

Wellness Allowance Management Portal Cognito Authentication & Authorization Integration

1. Introduction

1.1 Purpose

This document describes the **authentication and authorization architecture** used in the **Wellness Allowance Management Portal**. The system uses **AWS Cognito User Pools integrated with Google Workspace OAuth** to securely authenticate users and manage role-based access.

This document explains:

- Authentication flow
- Role-based access control
- AWS services used
- Role assignment logic
- Admin operations
- Security considerations

The goal of this architecture is to provide a **secure, scalable, and manageable identity system** for employees accessing the wellness allowance platform.

2. System Overview

The application allows employees within the organization to access a portal where wellness allowances are managed. The system restricts access to **Google Workspace accounts belonging to the organization's domain**.

Instead of managing passwords internally, authentication is delegated to **Google Identity Provider (IdP)** while **AWS Cognito manages user identities, tokens, and authorization groups**.

The system implements **Role-Based Access Control (RBAC)** where users are categorized into four groups:

- Employee
- HR
- Finance
- Admin

Each group determines what operations the user is allowed to perform inside the application.

3. System Architecture

The authentication architecture consists of the following major components.

Component	Description
Google Identity Provider	Handles user authentication using Google Workspace
AWS Cognito User Pool	Manages user identities, tokens, and groups
Cognito Hosted UI	Provides the login interface and handles OAuth redirection
Post-Confirmation Lambda	Assigns to groups when a user logs in for the first time and set verification status to 'pending'
AWS SSM Parameter Store	Stores Admin mails
Application Backend	Validates JWT tokens and performs authorization

4. User Roles and Access Model

The portal uses **four user roles**, implemented using **Cognito User Groups**.

Role	Description
Employee	Default user role

HR	Handles employee-related operations
Finance	Responsible for financial approvals and reporting
Admin	Manages user roles and verification

These roles(group) determine what parts of the system a user can access.

5. Authentication Flow

The system uses **OAuth 2.0 authorization code flow with Google as the Identity Provider**.

The authentication process begins when a user attempts to access the application.

Step 1 — User Access

The user navigates to the application. Since the user is not authenticated, the application redirects them to the **Cognito Hosted UI**.

Step 2 — Redirect to Google

Cognito redirects the user to **Google OAuth** for authentication.

The user logs in using their **Google Workspace account**.

Step 3 — Google Authentication

Google validates the user credentials and returns:

- OAuth access token
- User profile information
- Email address

This information is sent back to Cognito.

Step 4 — Hosted Domain Validation

Cognito checks the email domain of the authenticated user.

Only accounts belonging to the following domain are allowed:

@wealthos.cloud

If the domain does not match, the authentication request is rejected.

This ensures that only organization members can access the system.

Step 5 — User Creation (First Login)

If the user logs in for the first time:

1. Cognito creates a **federated user profile**.
2. The **Post-Confirmation Lambda trigger** is invoked.

The Lambda determines the group of the user.

Step 6 — Role/Group Assignment

We store admin mails in ssm parameters. When a user login if his mail is one of admin mails then user will add to admin group. Else users will add to employee group.

Step 7 — Token Issuance

After authentication is successful, Cognito issues a **JSON Web Token (JWT)**.

The JWT contains several claims including:

- user email
- Cognito ID
- assigned groups

Example token claim:

```
{
  "sub": "c1a23b45-6789-4abc-9f12-123456789abc",
  "email": "employee@wealthos.cloud",
  "name": "John Doe",
  "picture": "https://lh3.googleusercontent.com/...",
  "cognito:groups": ["employee"],
  "token_use": "id"
  "custom:verification_status": "verified"
}
```

Step 8 — Application Authorization

The user sends requests to the backend API using the JWT.

Example request header:

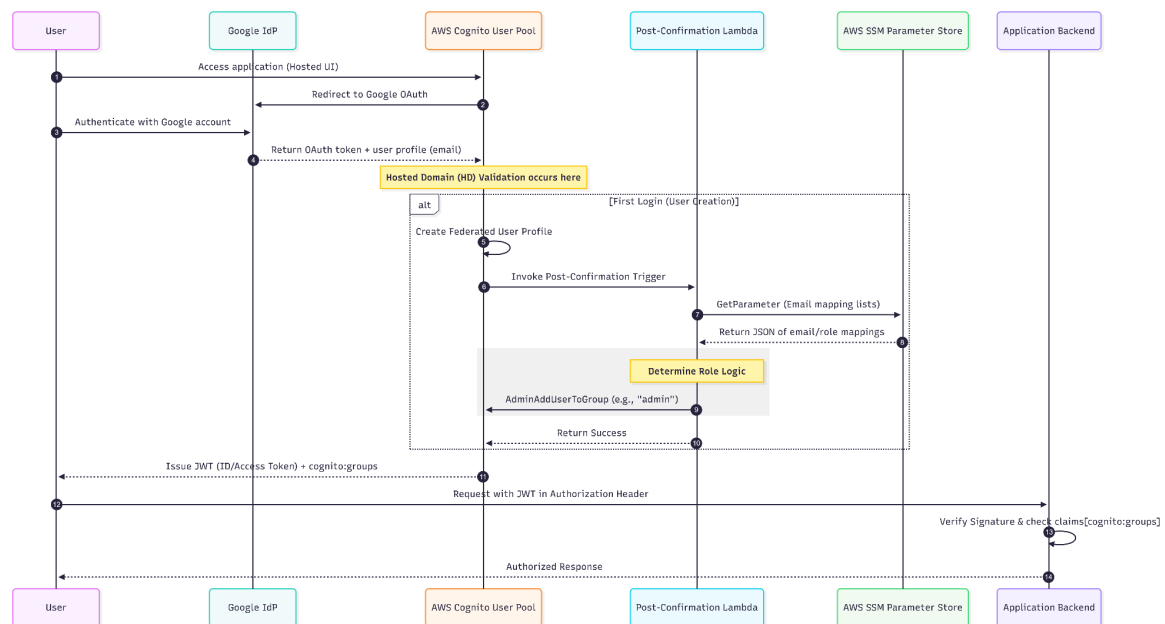
Authorization: Bearer <JWT_TOKEN>

The backend performs the following validations:

1. Verify token signature
 2. Check token expiration
 3. Extract the user's group information
 4. Apply role-based authorization rules
-

6. Authentication Sequence Diagram

Authentication sequence Diagram



8. Post-Confirmation Lambda Trigger

The **Post-Confirmation Lambda trigger** is executed immediately after a user successfully confirms their account for the first time.

Its responsibilities include:

1. Retrieve admin mails from SSM.
2. Check whether the user's email exists in a predefined list.
3. Assign the user to the appropriate Cognito group.

The function uses the following Cognito API: *AdminAddUserToGroup*

If the user is not found in the admin list, the user is assigned to the **employee group**.

9. Admin Capabilities

Users assigned to the **Admin role** have additional permissions within the system.

9.1 User Verification

Admins can verify users through a **custom Cognito attribute**.

Example attribute:

custom:verification_status

Possible values:

Value	Meaning
pending	Awaiting verification
verified	User approved

This attribute allows the system to implement an **additional verification layer** before granting access to certain operations.

9.2 Viewing Users

Admins can retrieve users grouped by their role.

Displayed information includes:

- User Name
- Email Address
- Profile Image
- Verification Status

This data is fetched from **Cognito user attributes**.

9.3 Managing User Roles

Admins can change a user's role by moving them between Cognito groups.

Examples include:

- Employee → HR
- Employee → Finance
- HR → Employee
- Finance → Employee

Role changes are implemented using the following APIs:

AdminAddUserToGroup

AdminRemoveUserFromGroup

10. Backend Authorization

The backend API is responsible for enforcing **access control policies**.

For every API request, the backend performs:

1. JWT validation using Cognito public keys.
2. Token expiration validation.
3. Authorization checks based on the **cognito:groups** claim.

Example claim:

```
{  
  "email": "user@wealthos.cloud",  
  "cognito:groups": ["finance"]  
}
```

